```
            ChemicalElement be = new ChemicalElement(2, 2, 's', 4, "Be");
            ChemicalElement mg = new ChemicalElement(2, 3, 's', 12, "Mg");
            ChemicalElement sc = new ChemicalElement(3, 4, 'd', 21, "Sc");
            ChemicalElement y = new ChemicalElement(3, 5, 'd', 39, "Y");
            ChemicalElement he = new ChemicalElement(18, 1, 'p', 2, "He");
            ChemicalElement la = new ChemicalElement(3, 6, 'f', 57, "La");
            TableOfChemichalElements gen = new TableOfChemichalElements();
            gen.add(h);
            gen.add(li);
            gen.add(na);
            gen.add(be);
            gen.add(mg);
            gen.add(sc);
            gen.add(y);
            gen.add(he);
            gen.add(la);
            return gen;
        }

        public static void main(String[] args) {

            TableOfChemichalElements gen = setUp();
            System.out.println(gen);
            TableOfChemichalElements group1 = gen.getGroup(1);
            System.out.println(group1);
            System.out.println(gen.getPeriod(1));
            System.out.println(gen.getSection('p'));

            System.out.println("----------------------------");
            Iterator it = gen.groupIterator(1);
            while (it.hasNext()) {
                ChemicalElement cE = (ChemicalElement) it.next();
                if (cE.getPeriod() == 2) {
                    it.remove();
                }
                System.out.println(cE);
            }
            System.out.println("----------------------------");
            System.out.println(gen);
        }
    }
```

Κώδικας 14.20 *Η κλάση TableOfChemichalElements*

Οι getGroup, getPeriod και getSection είναι προφανείς. Προκειμένου να υλοποιηθεί η getGroupIterator χρησιμοποιείται η εσωτερική κλάση GroupIterator η οποία υλοποιεί τη διεπαφή Iterator. Ο δημιουργός της κλάσης λαμβάνει ως παράμετρο το groupId της ομάδας του Iterator και ενημερώνει την αντίστοιχη μεταβλητή στιγμιότυπου. Επίσης, η ιδιωτική μεταβλητή current χρησιμεύει ως δείκτης στον πίνακα store της TableOfChemichalElements. Η hasNext αυξάνει την current έως ότου βρει στη θέση που υποδεικνύει η current στον πίνακα store στοιχείο που ανήκει στην ομάδα με groupId. Αν το βρει, επιστρέφει true διαφορετικά false. Η next επιστρέφει το στοιχείο στη θέση current του store και αυξάνει την current κατά 1. Η remove διαγράφει το στοιχείο που επέστρεψε η τελευταία κλήση της next. Ωστόσο, η next αφού επιστρέψει το στοιχείο αυξάνει την current κατά 1. Επομένως, η remove για να διαγράψει το σωστό στοιχείο θα πρέπει να μειώσει την current κατά 1. Εξάλλου με τη διαγραφή ενός στοιχείου μειώνεται και το μέγεθος του store κατά 1.

Έχοντας την κλάση GroupIterator, η συνάρτηση getGroupIterator επιστρέφει απλώς ένα αντικείμενο GroupIterator. Η setUp είναι στατική συνάρτηση που απλώς φορτώνει μερικά στοιχεία σε κατάλληλο ArrayList και τα επιστρέφει. Επίσης, κατάλληλα έχει υπερφορτωθεί και η toString ώστε να τυπώνει τα στοιχεία του πίνακα ένα ανά γραμμή βασιζόμενη στην toString της ChemicalElement.

Στην main φορτώνουμε με τη βοήθεια της setUp τον πίνακα gen, στη συνέχεια ελέγχουμε τις getGroup, getPeriod και getSection. Τέλος, ελέγχεται ο Iterator.

14.5.4 Τρίλιζα

Να υλοποιηθεί το γνωστό παιχνίδι τρίλιζα κατά το οποίο ο χρήστης θα παίζει με τον υπολογιστή. Να δίνεται η δυνατότητα στον χρήστη να επιλέγει ως αντίπαλο έναν τεχνητό παίκτη που επιλέγει τις κινήσεις του με βάση κανόνες (Rule based) ή έναν τεχνητό παίκτη που βασίζεται στον αλγόριθμο MinMax [4]. Επιπλέον, να υποστηριχθεί η δυνατότητα, οι δύο τεχνητοί παίκτες να παίζουν μεταξύ τους.

Το ταμπλό του παιχνιδιού να εμφανίζεται όπως φαίνεται στην εικόνα 14.1 και να ανανεώνεται με κάθε κίνηση.

```
  0 1 2          0 1 2
--------       --------
0|  |  |  |     0|X|O|  |
--------       --------
1|  |  |  |     1|  |  |  |
--------       --------
2|  |  |  |     2|  |  |  |
--------       --------
```

Εικόνα 14.1 *Το ταμπλό της τρίλιζας στην αρχή του παιχνιδιού και μετά από 2 κινήσεις.*

Ο Rule based παίκτης να επιλέγει την κίνησή του με βάση τους ακόλουθους κανόνες:

1. Αν υπάρχει κίνηση με την οποία κάνει τρίλιζα την επιλέγει.
2. Αν ο αντίπαλός του έχει κίνηση με την οποία μπορεί να κάνει τρίλιζα στην επόμενη κίνηση, ο rule based επιλέγει αυτήν την κίνηση ώστε να μπλοκάρει τη νικηφόρα κίνηση του αντιπάλου.
3. Επιλέγει την πρώτη διαθέσιμη κίνηση.

Ο Minimax ως αλγόριθμος μπορεί να χρησιμοποιηθεί σε πολλά παιχνίδια για να εντοπίσει τη βέλτιστη κίνηση. Εκείνο που κάνει είναι ότι υπολογίζει το σύνολο των δυνατών παρτίδων και επιλέγει μονοπάτια που δεν οδηγούν σε ήττα. Ο τεχνητός παίκτης που βασίζεται στον Minimax είναι ένας τέλειος παίκτης και δεν χάνει ποτέ. Ωστόσο, αν αντιμετωπίσει έναν επίσης δυνατό παίκτη μπορεί να οδηγηθεί σε ισοπαλία. Ο αναγνώστης που ενδιαφέρεται για περισσότερες λεπτομέρειες μπορεί να παρακολουθήσει την εξαιρετική διάλεξη του Patrick Winston από το MIT [4].

Ο κώδικας να σχολιαστεί κατάλληλα ώστε να υποστηρίζει την παραγωγή τεκμηρίωσης java.

Λύση

Η λύση που παρατίθεται εδώ συντίθεται από επτά κλάσεις ως εξής: Η αφηρημένη κλάση Player και οι παράγωγές της σαφείς κλάσεις LPlayer μέσω της οποίας ο άνθρωπος παίζει τρίλιζα με τον υπολογιστή και οι τεχνητοί παίκτες RBPlayer (Rule based παίκτης), MinMaxPlayer (τεχνητός παίκτης που βασίζεται στον MiniMax αλγόριθμο), η βοηθητική κλάση Movement που αναπαριστά μία κίνηση στο ταμπλό, η κλάση Board που αναπαριστά το ταμπλό και η κλάση Game που οργανώνει τις υπόλοιπες και περιλαμβάνει την main της εφαρμογής.

Στη συνέχεια παρατίθεται ο κώδικας με κατάλληλα σχόλια από τα οποία αυτόματα μπορεί να εξαχθεί η τεκμηρίωση της εφαρμογής.

```
/**
 * Αναπαριστά το ταμπλό της Τρίλιζας
 *
 * @author Lefteris Moussiades
 */
public class Board {
```

```
private final char[][] board;

/**
 * Σημειώνει τις άδειες θέσεις στο ταμπλό
 */
public static final char EMPTYCELL = ' ';

/**
 * Ο Δημιουργός της κλάσης. Δημιουργεί ένα ταμπλό Τρίλιζας. Όλες οι
 * θέσεις αρχικοποιούνται στο EMPTYCELL
 */
public Board() {
    board = new char[3][3];
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            board[i][j] = EMPTYCELL;
        }
    }
}

/**
 * Δημιουργεί ένα αντίγραφο του ταμπλό
 *
 * @param source Το ταμπλό του οποίου θα δημιουργηθεί αντίγραφο
 */
public Board(Board source) {
    board = new char[3][3];
    for (int i = 0; i < 3; i++) {
        System.arraycopy(source.board[i], 0, board[i], 0, 3);
    }
}

/**
 * Δημιουργεί αυτόματα μια κίνηση για τον παίκτη p. Καλείται μόνο εφόσον
 * υπάρχει μια μόνο θέση ελεύθερη στο ταμπλό και είναι σειρά του παίκτη
 * p.
 *
 */
void autoMove(Player p) {
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (board[i][j] == EMPTYCELL) {
                board[i][j] = p.getId();
            }
        }
    }
}

/**
 * Εμφανίζει το ταμπλό
 */
void displayBoard() {
    System.out.println("  0 1 2");
    for (int i = 0; i < 3; i++) {
        System.out.println("--------");
        System.out.print(i);
        for (int j = 0; j < 3; j++) {
            System.out.print("|" + board[i][j]);
        }
        System.out.println("|");
```

```
        }
        System.out.println("--------");
    }

    /**
     * Θέτει την κίνηση mv του παίκτη p στο ταμπλό
     *
     */
    public void setPlayerMove(Player p, Movement mv) {
        board[mv.getRow()][mv.getClmn()] = p.getId();
    }

    /**
     *
     * @return τον αριθμό των ελεύθερων θέσεων στο ταμπλό
     */
    public int freePositions() {
        int cnt = 0;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (board[i][j] == EMPTYCELL) {
                    cnt++;
                }
            }
        }
        return cnt;
    }

    private char lineWin(int ln) {
        if ((board[ln][0] != EMPTYCELL)
                && (board[ln][0] == board[ln][1])
                && (board[ln][0] == board[ln][2])) {
            return board[ln][0];
        }
        return EMPTYCELL;
    }

    private char columnWin(int clmn) {
        if ((board[0][clmn] != EMPTYCELL)
                && (board[0][clmn] == board[1][clmn])
                && (board[1][clmn] == board[2][clmn])) {
            return board[0][clmn];
        }
        return EMPTYCELL;
    }

    private char diagonalWin() {
        if ((board[0][0] != EMPTYCELL)
                && (board[0][0] == board[1][1])
                && (board[0][0] == board[2][2])) {
            return board[0][0];
        }
        if ((board[2][0] != EMPTYCELL)
                && (board[2][0] == board[1][1])
                && (board[2][0] == board[0][2])) {
            return board[2][0];
        }
        return EMPTYCELL;
    }

    /**
```

```
     *
     * @return την ταυτότητα του νικητή. Αν δεν υπάρχει νικητής επιστρέφει
     * EMPTYCELL;
     */
    public char win() {
        char rslt;
        for (int i = 0; i < 3; i++) {
            if ((rslt = lineWin(i)) != EMPTYCELL) {
                return rslt;
            }
            if ((rslt = columnWin(i)) != EMPTYCELL) {
                return rslt;
            }
        }
        return diagonalWin();
    }

    /**
     *
     * @return έναν πίνακα που περιέχει τις ελεύθερες θέσεις του ταμπλό
     */
    public Movement[] possibleMoves() {
        Movement[] rVal;
        int cnt = 0;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (board[i][j] == EMPTYCELL) {
                    cnt++;
                }
            }
        }
        if (cnt == 0) {
            return null;
        }
        rVal = new Movement[cnt];

        cnt = 0;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (board[i][j] == EMPTYCELL) {
                    rVal[cnt++] = new Movement(i, j);
                }
            }
        }
        return rVal;
    }
}
```

Κώδικας 14.21 *Η κλάση Board*

```
/**
 * Αναπαριστά μία κίνηση στο ταμπλό της Τρίλιζας. Μία κίνηση αναπαρίσταται
 * από δύο ακέραιους, τη γραμμή και τη στήλη στο ταμπλό. Τα αντικείμενα της
 * κλάσης είναι αμετάβλητα.
 *
 * @author Lefteris Moussiades
 */
public class Movement {

    private final int r;
```

```
    private final int c;

    public Movement(int row, int clmn) {
        r = row;
        c = clmn;
    }

    public int getRow() {
        return r;
    }

    public int getClmn() {
        return c;
    }
}
```

Κώδικας 14.22 *Η κλάση Movement*

```
/**
 * Αφηρημένη κλάση που αναπαριστά έναν παίκτη Τρίλιζας. Όλες οι σαφείς κλάσεις
 * που αναπαριστούν παίκτες Τρίλιζας πρέπει να είναι παράγωγες της Player.
 */
public abstract class Player {

    private final char id;
    private Player opponent;

    /**
     * Constructor
     *
     * @param id Η ταυτότητα του παίκτη, π.χ. x or o etc. Χρησιμοποιείται
     * επιπλέον κατά την επισήμανση των θέσεων του παίκτη στο ταμπλό
     *
     */
    public Player(char id) {
        this.id = id;
    }

    public char getId() {
        return id;
    }

    /**
     *
     * @param board
     * @return true Αν αυτός ο παίκτης έχει νικήσει
     */
    public boolean wins(Board board) {
        char wSign = board.win();
        return wSign == id;
    }

    /**
     * Ενημερώνει αυτόν τον παίκτη με τον αντίπαλό του. Ενημερώνει τον
     * αντίπαλο
     * με αυτόν τον παίκτη
     *
     * @param p the opponent Player
     */
    public void setOpponents(Player p) {
```

```
        opponent = p;
        p.opponent = this;
    }

    public Player getOpponent() {
        return opponent;
    }

    /**
     * Όλες οι σαφείς κλάσεις παικτών Τρίλιζας πρέπει να υλοποιούν αυτήν τη
     * μέθοδο
     *
     * @param par Λαμβάνει ως παράμετρο το ταμπλό της Τρίλιζας στην τρέχουσα
     * κατάσταση
     * @return Με βάση την τρέχουσα κατάσταση του ταμπλό, ο παίκτης
     * "αποφασίζει" την κίνησή του. Επιστρέφει αντικείμενο Movement
     */
    public abstract Movement move(Board par);

}
```

Κώδικας 14.23 *Η αφηρημένη κλάση Player*

```
/**
 * Represents ένα ζώντα παίκτη, δηλαδή έναν παίκτη που δίνει την κίνησή του από
 * το πληκτρολόγιο
 *
 */
public class LPlayer extends Player {

    public LPlayer(char id) {
        super(id);
    }

    /**
     * Λαμβάνει από το πληκτρολόγιο δύο ακέραιες τιμές
     *
     * @param board
     * @return Μια κίνηση (Movement)
     */
    @Override
    public Movement move(Board board) {
        Scanner cIn = new Scanner(System.in);
        int r, c;
        System.out.print("dwse grammi : ");
        r = cIn.nextInt();
        System.out.println();
        System.out.print("dwse stili : ");
        c = cIn.nextInt();
        return new Movement(r, c);
    }

}
```

Κώδικας 14.23 *Η κλάση LPlayer, παράγωγη της Player*

```
/**
 * It Represents a RuleBased Player
 *
 */
public class RBPlayer extends Player {
```

```
    public RBPlayer(char id) {
        super(id);
    }

    /**
     * Υλοποιεί την αφηρημένη μέθοδο της κλάσης Player σύμφωνα με τις
     * απαιτούμενες προδιαγραφές για τον Rule Based παίκτη
     *
     * @param board
     * @return Μία κίνηση (Movement)
     */
    @Override
    public Movement move(Board board) {
        Movement[] availM = board.possibleMoves();
        for (Movement mv : availM) {
            Board bd = new Board(board);
            bd.setPlayerMove(this, mv);
            if (getId() == bd.win()) {
                return mv;
            }
        }
        for (Movement mv : availM) {
            Board bd = new Board(board);
            bd.setPlayerMove(getOpponent(), mv);
            if (getOpponent().getId() == bd.win()) {
                return mv;
            }
        }
        return availM[0];
    }
}
```

Κώδικας 14.24 *Η κλάση RBPlayer, παράγωγη της Player*

```
/**
 * Κληρονομεί την αφηρημένη κλάση Player. Υλοποιεί τον αλγόριθμο minimax Ο
 * MiniMaxPlayer είναι τέλειος παίκτης. Δεν χάνει ποτέ.
 *
 * @author Lefteris
 */
public class MiniMaxPlayer extends Player {

    Movement currentMovement;

    public MiniMaxPlayer(char id) {
        super(id);
    }

    private int maxIdx(int[] tbl) {
        int rVal = 0, max = tbl[0];
        for (int i = 1; i < tbl.length; i++) {
            if (tbl[i] > max) {
                max = tbl[i];
                rVal = i;
            }
        }
        return rVal;
    }

    private int minIdx(int[] tbl) {
        int rVal = 0, min = tbl[0];
```

```
        for (int i = 1; i < tbl.length; i++) {
            if (tbl[i] < min) {
                min = tbl[i];
                rVal = i;
            }
        }
        return rVal;
    }

    private int minimax(Board board, Player p) {
        if (getOpponent().wins(board)) {
            return -1;
        }

        if (wins(board)) {
            return 1;
        }
        if (board.freePositions() == 0) {
            return 0;
        }
        Movement[] moves = board.possibleMoves();
        int[] scores = new int[moves.length];

        for (int i = 0; i < moves.length; i++) {
            Board newState = new Board(board);
            newState.setPlayerMove(p, moves[i]);
            scores[i] = minimax(newState, p.getOpponent());
        }

        if (this.equals(p)) {
            int maxScoreIdx = maxIdx(scores);
            currentMovement = moves[maxScoreIdx];
            return scores[maxScoreIdx];
        } else {
            int minScoreIdx = minIdx(scores);
            currentMovement = moves[minScoreIdx];
            return scores[minScoreIdx];
        }
    }

    /**
     *
     * @param board
     * @return Κίνηση με βάση τον minimax
     */
    @Override
    public Movement move(Board board) {
        minimax(board, this);
        return currentMovement;
    }
};
```

Κώδικας 14.25 *Η κλάση MiniMaxPlayer, παράγωγη της Player*

```
public class Game {

    Board board = new Board();
    Player p1;
    Player p2;

    /**
     * Δημιουργεί ένα παιχνίδι αποτελούμενο από ένα ταμπλό και δύο παίκτες
```

```
     * Ενημερώνει κάθε παίκτη με τον αντίπαλό του.
     *
     * @param p1 Player 1, αντίπαλος του Player 2
     * @param p2 Player 2, αντίπαλος του Player 1
     */
    Game(Player p1, Player p2) {
        this.p1 = p1;
        this.p2 = p2;
        p1.setOpponents(p2);
    }

    /**
     * Με βάση την board.win() ελέγχει αν υπάρχει νικητής
     *
     * @return Ο παίκτης που νίκησε ή null αν δεν υπάρχει νικητής
     * @see gr.ihu.cs.lmous.TicTacToe.Board#win()
     */
    Player winner() {
        char winnerSign = board.win();
        if (p1.getId() == winnerSign) {
            return p1;
        }
        if (p2.getId() == winnerSign) {
            return p2;
        }
        return null;
    }

    /**
     * Εκκινεί ένα παιχνίδι
     *
     * @param p Ο παίκτης που παίζει πρώτος
     * @return Τον νικητή ή null σε περίπτωση ισοπαλίας
     */
    Player play(Player p) {
        Player theWinner = null;
        Player current = p;

        do {
            Movement cell;
            if (board.freePositions() == 1) {
                board.autoMove(current);
            } else {
                cell = current.move(board);
                board.setPlayerMove(current, cell);
            }
            board.displayBoard();
            current = current.getOpponent();

        } while (board.freePositions() > 0
                && (theWinner = winner()) == null);

        return theWinner;
    }

    public static void main(String[] args)/*throws Exception*/ {
        Player p2 = new MiniMaxPlayer('X');
        Player p1 = new LPlayer('O');
        Player p3 = new RBPlayer('r');

        Game myGame = new Game(p2, p3);
```

```
        myGame.board.displayBoard();
        Player winner = myGame.play(p2);

        if (winner != null) {
            System.out.println("Winner is " + winner.getId());
        } else {
            System.out.println("withdraw ");
        }
    }
}
```

Κώδικας 14.26 *Η κλάση Game οργανώνει μια παρτίδα τρίλιζα*

14.6 Ασκήσεις προς Λύση

14.6.1 Πολλαπλές τράπουλες

Να υλοποιηθεί η κλάση AnyDeck, τα αντικείμενα της οποίας είναι τράπουλες που περιέχουν κάρτες στα όρια βαθμών (CardRank). Πιο συγκεκριμένα, ο δημιουργός της κλάσης έχει τη μορφή public AnyDeck(CardRank from, CardRank to) και δημιουργεί μια τράπουλα με όλες κάρτες από βαθμό from έως και βαθμό to. Να υλοποιηθεί η εφαρμογή της άσκησης 14.5.1 με τη βοήθεια δύο αντικειμένων της AnyDeck.

14.6.2 Μελέτη

Στην άσκηση 14.5.1, οι κλάσεις Deck52 και Deck32 έχουν πάρα πολλές ομοιότητες μεταξύ τους. Πράγματι, αν μελετήσετε τις δύο κλάσεις, θα διαπιστώσετε πως οι περισσότερες συναρτήσεις είναι σχεδόν ίδιες. Είναι εφικτό η μία κλάση να ορισθεί παράγωγη της άλλης ώστε να κληρονομήσει τις μεθόδους και να μην τις ξαναγράφουμε; Αν ναι, προχωρήστε σε αντίστοιχη τροποποίηση. Αν όχι, δώστε μια διαφορετική λύση που ενώ διατηρεί τις 2 κλάσεις, αποφεύγει την επανάληψη του κώδικα κατά το δυνατόν.

14.6.3 Περιοδικός Πίνακας

Υλοποιήστε την κλάση PeriodTable. Η κλάση αναπαριστά τον περιοδικό πίνακα των χημικών στοιχείων. Η κλάση για κάθε στοιχείο του περιοδικού πίνακα, περιέχει ένα αντικείμενο ChemicalElement που μπορούμε να προσπελάσουμε με τη συνάρτηση μέλος της κλάσης, ChemicalElement getElement(String symbol). Η κλάση δεν επιτρέπει εισαγωγές και διαγραφές στοιχείων, ενώ υλοποιεί Iterators, έναν για τις ομάδες, έναν για τις περιόδους και έναν για τους τομείς.

Βιβλιογραφία

[1] “Interfaces (The Java™ Tutorials > Learning the Java Language > Interfaces and Inheritance).” https://docs.oracle.com/javase/tutorial/java/IandI/createinterface.html (accessed Oct. 27, 2021).

[2] “Multiple Inheritance of State, Implementation, and Type (The Java™ Tutorials > Learning the Java Language > Interfaces and Inheritance).” https://docs.oracle.com/javase/tutorial/java/IandI/multipleinheritance.html (accessed Oct. 27, 2021).

[3] “Περιοδικός πίνακας των χημικών στοιχείων,” Βικιπαίδεια. Oct. 23, 2021. Accessed: Nov. 02, 2021. [Online]. Available: https://el.wikipedia.org/w/index.php?title Περιοδικός_πίνακας_των_χημικών_στοιχείων&oldid=9105038

[4] “Minimax,” Wikipedia. Oct. 05, 2021. Accessed: Nov. 01, 2021. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Minimax&oldid=1048398851

[5] MIT OpenCourseWare, 6. Search: Games, Minimax, and Alpha-Beta, (Jan. 10, 2014). Accessed: Nov. 01, 2021. [Online Video]. Available: https://www.youtube.com/watch?v=STjW3eH0Cik